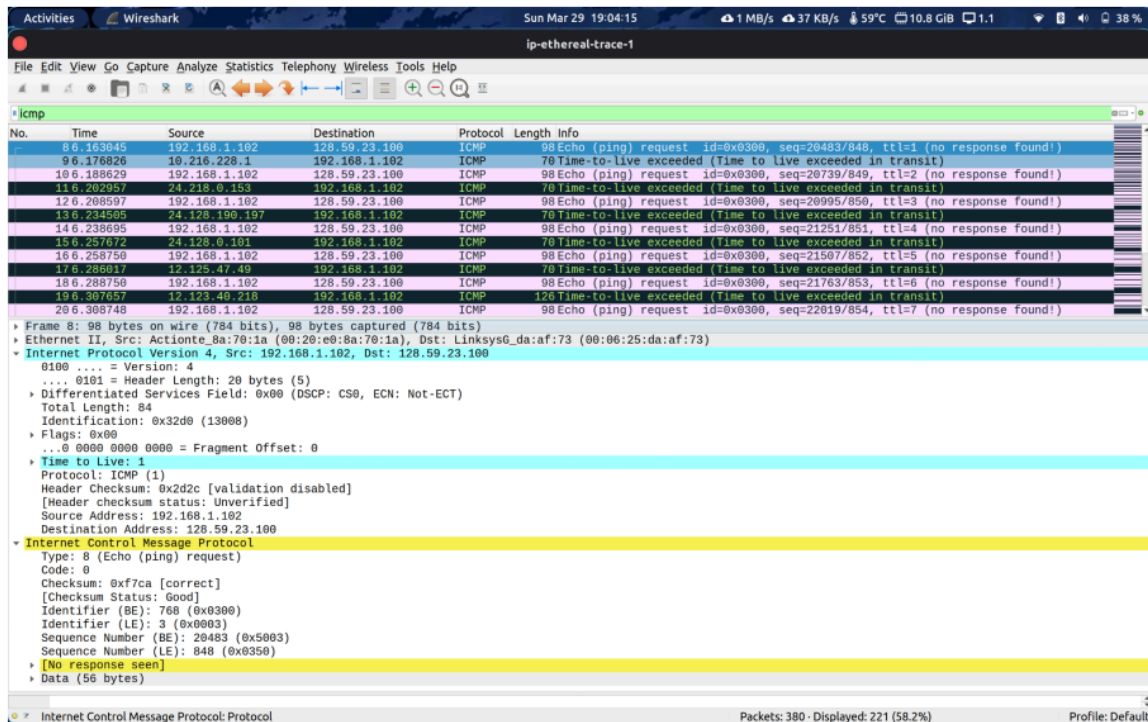


1. Question 1

Q1.1. Part A - IP Wireshark Lab

1.1.1 What is the IP address of your computer?

The source IP address seen in the ICMP Echo Request packets is **192.168.1.102**.



1.1.2 What is the value in the upper layer protocol field?

The Protocol field in the IP header reads **ICMP (Protocol Number = 1)**.

1.1.3 How many bytes are in the IP header and payload?

- **IP Header:** 20 bytes (IHL = 5, so $5 \times 4 = 20$ bytes)
- **Total Length:** 84 bytes
- **Payload:** $84 - 20 = 64$ bytes

The payload size is obtained by subtracting the header length from the Total Length field, since Total Length covers both header and data.

1.1.4 Has this IP datagram been fragmented?

No. The More Fragments (MF) bit is 0 and the Fragment Offset is 0, which together confirm the datagram arrived as a single, unfragmented unit.

1.1.5 Which fields change from one datagram to the next?

- **TTL** - traceroute deliberately increments this to probe each hop.
- **Identification** - assigned a fresh value for every new datagram.
- **Header Checksum** - recomputed whenever any header field changes.

1.1.6 Which fields stay constant? Which must change?

Field	Status	Reason
Source IP	Constant	Identifies the sending host; must stay the same throughout.
Destination IP	Constant	Identifies the target; must remain fixed.
Version	Constant	IPv4 throughout.
Header Length	Constant	Fixed at 20 bytes (no options).
Protocol	Constant	All packets carry ICMP.
TTL	Changes	Traceroute increments it to find each router hop.
Identification	Changes	Must be unique per datagram for correct re-assembly.
Header Checksum	Changes	Recalculated after every header modification.

Fields defining the communication endpoints (Source IP, Destination IP, Protocol) must be constant; otherwise packets would not belong to the same flow.

1.1.7 Describe the pattern in the Identification field.

The Identification field increments by 1 with each successive datagram, giving every packet a unique identifier. This sequential numbering is important for correct fragment reassembly if the datagram is later split.

1.1.8 What are the Identification and TTL values in the first TTL-exceeded reply?

- **Identification:** 0x9d7c (decimal 40316)
- **TTL:** 255

1.1.9 Do these values stay the same for all TTL-exceeded replies from the first hop?

- **TTL** remains constant at 255 for all replies from the same router, because this value is set by the router's operating system and does not vary between replies.
- **Identification** changes with each reply, because every ICMP Time Exceeded message is a brand-new IP datagram generated by the router and therefore receives a fresh Identification number.

1.1.10 Has the 2000-byte ICMP Echo Request been fragmented?

Yes. The datagram (Identification 0x32f9) was split into two fragments:

[2 IPv4 Fragments (2008 bytes): #92(1480), #93(528)]

Both fragments share the same Identification value and carry different Fragment Offset values, confirming fragmentation occurred.

Time	Source	No.	Destination	Protocol	Length	Info
28.470668	192.168.1.102	95	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fa) [Reassembled in #96]
28.471338	192.168.1.102	96	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=30723/888, ttl=2 (no response found!)
28.490663	192.168.1.102	97	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fb) [Reassembled in #98]
28.491323	192.168.1.102	98	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=30979/889, ttl=3 (no response found!)
28.520729	192.168.1.102	99	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fc) [Reassembled in #100]
28.521393	192.168.1.102	100	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31235/890, ttl=4 (no response found!)
28.530213	24.218.0.153	101	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
28.540758	192.168.1.102	102	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fd) [Reassembled in #103]
28.541476	192.168.1.102	103	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31491/891, ttl=5 (no response found!)
28.570848	192.168.1.102	104	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fe) [Reassembled in #105]
28.571603	192.168.1.102	105	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31747/892, ttl=6 (no response found!)
28.590801	192.168.1.102	106	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32ff) [Reassembled in #107]
28.591465	192.168.1.102	107	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=32003/893, ttl=7 (no response found!)

▶ Frame 95: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits)
 ▶ Ethernet II, Src: Actionte_8a:70:1a (00:20:e0:8a:70:1a), Dst: Linksys6_da:af:73 (00:06:25:da:af:73)
 ▶ Internet Protocol Version 4, Src: 192.168.1.102, Dst: 128.59.23.100
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 Total Length: 1500
 Identification: 0x32fa (13050)
 ▶ Flags: 0x20, More Fragments
 ...0 0000 0000 0000 = Fragment Offset: 0
 ▶ Time to Live: 2
 Protocol: ICMP (1)
 Header Checksum: 0x067a [validation disabled]
 [Header checksum status: Unverified]
 Source Address: 192.168.1.102
 Destination Address: 128.59.23.100
 [Reassembled IPv4 in frame: 96]
 ▶ Data (1480 bytes)
 Data: 0809cfc603097803373620aa...
 [Length: 1480]

1.1.11 Describe the first fragment.

The datagram is fragmented because:

- The **More Fragments (MF) flag** = 1, indicating at least one more fragment follows.
- Multiple packets share the same Identification value.

This is the **first** fragment because its **Fragment Offset** = 0 (starts at the beginning of the original data) and MF = 1. Its total size at the IP layer is **1500 bytes** (20-byte header + 1480-byte payload). The complete original datagram totals **2008 bytes**.

1.1.12 Describe the second fragment.

The second fragment has **Fragment Offset** = 1480 (non-zero), which immediately shows it is *not* the first fragment. Its **MF flag** = 0, meaning no further fragments follow - this is the final piece. Its total size is 548 bytes (20-byte header + 528-byte payload).

Time	Source	No.	Destination	Protocol	Length	Info
28.470668	192.168.1.102	95	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fa) [Reassembled in #96]
28.471338	192.168.1.102	96	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=30723/888, ttl=2 (no response found!)
28.490663	192.168.1.102	97	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fb) [Reassembled in #98]
28.491323	192.168.1.102	98	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=30979/889, ttl=3 (no response found!)
28.520729	192.168.1.102	99	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fc) [Reassembled in #100]
28.521393	192.168.1.102	100	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31235/890, ttl=4 (no response found!)
28.530213	24.218.0.153	101	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
28.540758	192.168.1.102	102	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fd) [Reassembled in #103]
28.541476	192.168.1.102	103	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31491/891, ttl=5 (no response found!)
28.570848	192.168.1.102	104	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32fe) [Reassembled in #105]
28.571603	192.168.1.102	105	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=31747/892, ttl=6 (no response found!)
28.590801	192.168.1.102	106	128.59.23.100	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=32ff) [Reassembled in #107]
28.591465	192.168.1.102	107	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=32003/893, ttl=7 (no response found!)

▶ Frame 96: 562 bytes on wire (4496 bits), 562 bytes captured (4496 bits)
 ▶ Ethernet II, Src: Actionte_8a:70:1a (00:20:e0:8a:70:1a), Dst: Linksys6_da:af:73 (00:06:25:da:af:73)
 ▶ Internet Protocol Version 4, Src: 192.168.1.102, Dst: 128.59.23.100
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 Total Length: 548
 Identification: 0x32fa (13050)
 Flags: 0x00
 ...0 0101 1100 1000 = Fragment Offset: 1480
 ▶ Time to Live: 2
 Protocol: ICMP (1)
 Header Checksum: 0x2979 [validation disabled]
 [Header checksum status: Unverified]
 Source Address: 192.168.1.102
 Destination Address: 128.59.23.100
 ▶ 2 IPv4 Fragments (2008 bytes): #95(1480), #96(528)]
 [Frame 95, payload: 0-1479 (1480 bytes)]
 [Frame 96, payload: 1480-2007 (528 bytes)]
 [Fragment count: 2]
 [Reassembled IPv4 length: 2008]
 [Reassembled IPv4 data: 0809cfc603097803373620aa...]

1.1.13 Which fields change between the first and second fragment?

- **Fragment Offset** - 0 in the first fragment, 1480 in the second.
- **More Fragments (MF) flag** - 1 in the first, 0 in the second.
- **Total Length** - 1500 bytes vs. 548 bytes.

- **Header Checksum** - recalculated for each fragment because header fields differ.

1.1.14 How many fragments were created for the larger datagram?

The original datagram was split into **3 fragments**:

[3 IPv4 Fragments (3508 bytes): #216(1480), #217(1480), #218(548)]

1.1.15 Which fields change among the three fragments?

- **Fragment Offset** - 0, 1480, and 2960 for fragments 1, 2, and 3 respectively.
- **More Fragments (MF) flag** - set to 1 for the first two, cleared to 0 for the last.
- **Total Length** - varies per fragment (1500, 1500, and 568 bytes).
- **Header Checksum** - recalculated for each fragment.

Time	Source	No.	Destination	Protocol	Length	Info
38.617611	24.128.190.197	191	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
38.635988	192.168.1.102	193	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=38915/920, ttl=8 (no response found!)
38.655981	192.168.1.102	195	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=39171/921, ttl=9 (no response found!)
38.668043	192.168.1.102	197	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=39427/922, ttl=10 (no response found!)
38.691895	24.128.0.107	198	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
38.706155	192.168.1.102	200	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=39683/923, ttl=11 (no response found!)
38.736256	192.168.1.102	202	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=39939/924, ttl=12 (no response found!)
38.755345	12.125.47.49	203	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
38.756348	192.168.1.102	205	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=40195/925, ttl=13 (reply in 214)
39.164169	67.99.58.194	211	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
39.227649	128.59.1.41	212	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
43.467629	192.168.1.102	218	128.59.23.100	ICMP	562	Echo (ping) request id=0x0300, seq=40451/926, ttl=1 (no response found!)
43.485786	10.216.228.1	219	192.168.1.102	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

* Frame 218: 582 bytes on wire (4656 bits), 582 bytes captured (4656 bits)
 * Ethernet II, Src: Actionte_Ba:70:1a (08:20:e0:8a:70:1a), Dst: Linksys6_da:af:73 (08:06:25:da:af:73)
 * Internet Protocol Version 4, Src: 192.168.1.102, Dst: 128.59.23.100
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 * Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 Total Length: 568
 Identification: 0x3323 (13091)
 Flags: 0x01
 ... 0 1011 1001 0000 = Fragment Offset: 2960
 * Time to Live: 1
 Protocol: ICMP (1)
 Header Checksum: 0x2983 [validation disabled]
 [Header checksum status: Unverified]
 Source Address: 192.168.1.102
 Destination Address: 128.59.23.100
 * [3 IPv4 Fragments (3508 bytes): #216(1480), #217(1480), #218(548)]
 [Frame 216, payload: 0-1479 (1480 bytes)]
 [Frame 217, payload: 1480-2959 (1480 bytes)]
 [Frame 218, payload: 2960-3507 (548 bytes)]
 [Fragment count: 3]
 [Reassembled IPv4 length: 3508]
 [Reassembled IPv4 data: 0800a9c303009e03373929aa...]

Q1.2. Part B - ICMP Wireshark Lab

1.2.1 Source and destination IP addresses.

- **Source (your host):** 192.168.1.101
- **Destination:** 143.89.14.34

1.2.2 Why does ICMP have no port numbers?

ICMP operates at the **Network Layer (Layer 3)** and is used for error reporting and diagnostics between routers and hosts, not for application-to-application communication. Port numbers exist at the Transport Layer to distinguish processes on a host. Since ICMP has no such role, port numbers are unnecessary.

1.2.3 ICMP Echo Request fields.

- **Type: 8** (Echo Request), **Code: 0**

Additional fields and their sizes:

- Type (1 byte), Code (1 byte), Checksum (2 bytes), Identifier (2 bytes), Sequence Number (2 bytes), Data/Payload (variable)

1.2.4 ICMP Echo Reply fields.

- **Type: 0** (Echo Reply), **Code: 0**

The reply carries the same fields as the request: Type (1 byte), Code (1 byte), Checksum (2 bytes), Identifier (2 bytes), Sequence Number (2 bytes), and the original data payload.

1.2.5 Source and destination for the traceroute capture.

- **Source:** 192.168.1.101
- **Destination:** 138.96.146.2

1.2.6 Would the IP protocol number change if UDP were used?

Yes. Protocol number 1 is reserved for ICMP. If traceroute used UDP probes (as is common on Unix/Linux), the IP protocol number would be **17** (UDP).

1.2.7 Is the traceroute ICMP packet different from a ping packet?

Yes. The key difference lies in the **TTL field**:

- **Traceroute:** TTL is intentionally set to a small value (1, 2, 3, ...) so that intermediate routers discard the packet and return an ICMP Time Exceeded message, revealing each hop along the path.
- **Ping:** TTL is set large (e.g., 128) so the packet reaches the destination.

The ICMP type and structure are identical; only the TTL strategy differs.

1.2.8 What extra fields does the ICMP error packet contain?

An ICMP Time Exceeded error packet includes, beyond the standard ICMP header:

- The **IP header of the original packet** (source, destination, TTL, Protocol, etc.).
- The **first 8 bytes of the original packet's payload** (enough to include the original ICMP Identifier and Sequence Number).

This embedded information lets the sender match each error back to the specific probe that triggered it.

1.2.9 How do the last three ICMP packets differ from the error packets?

The final three packets are **Echo Reply (Type 0, Code 0)** messages sent by the **destination host** itself when the probe packet finally reaches it. The earlier packets are **Time Exceeded (Type 11, Code 0)** error messages sent by **intermediate routers** each time a probe's TTL expires before reaching the destination.

1.2.10 Is there a link with notably higher delay? Can you identify its endpoints?

Yes. There is a significant delay jump between **hop 9 and hop 10**, where round-trip time increases from roughly 25 ms to about 98 ms. Based on the router hostnames in the trace:

- **Hop 9:** att-gw.nyc.opentransit.net - New York, USA.
- **Hop 10:** Pastourelle.opentransit.net - France, Europe.

The large delay is consistent with a **transatlantic submarine cable link**, which introduces substantially higher propagation delay than domestic links.

2. Question 2

Network topology: 4 nodes (0–3) with bi-directional link costs $c(0,1) = 1$, $c(0,2) = 3$, $c(0,3) = 7$, $c(1,2) = 1$, $c(2,3) = 2$. Nodes 1 and 3 are not directly connected.

Q2.1. Part 1: Link-State Routing (Dijkstra's Algorithm)

2.1.1 Algorithm Description

Link-State routing is a **global** algorithm: every node has full knowledge of the topology and independently runs Dijkstra's shortest-path algorithm. The key quantities are:

- $D(v)$ - current best-known cost from source to v .
- N' - set of nodes whose shortest path has been confirmed.
- Update rule: $D(v) \leftarrow \min(D(v), D(u) + c(u, v))$.

2.1.2 Python Implementation

Listing 1: Link-State Routing via Dijkstra's Algorithm

```
1  import heapq
2
3  graph = {
4      0: {1: 1, 2: 3, 3: 7},
5      1: {0: 1, 2: 1},
6      2: {0: 3, 1: 1, 3: 2},
7      3: {0: 7, 2: 2},
8  }
9
10 NODES = list(graph.keys())
11 INF = float('inf')
12
13 def dijkstra(source):
14     dist = {n: INF for n in NODES}
15     next_hop = {n: None for n in NODES}
16     dist[source] = 0
17     pq = [(0, source)]
18
19     while pq:
20         cost, u = heapq.heappop(pq)
21         if cost > dist[u]:
22             continue
23         for v, w in graph[u].items():
24             new_cost = dist[u] + w
25             if new_cost < dist[v]:
26                 dist[v] = new_cost
27                 next_hop[v] = v if u == source else next_hop[u]
28                 heapq.heappush(pq, (new_cost, v))
29
30     return dist, next_hop
```

2.1.3 Dijkstra Step-by-Step Tables

Bold entries denote the node added to N' at each step.

Source Node 0

Step	N'	$D(0)$	$D(1)$	$D(2)$	$D(3)$
Init	{0}	0	(1,1)	(3,2)	(7,3)
1	{0,1}	0	(1,1)	(2,1)	(7,3)
2	{0,1,2}	0	(1,1)	(2,1)	(4,1)
3	{0,1,2,3}	0	(1,1)	(2,1)	(4,1)

Paths: $0 \rightarrow 1$ (cost 1); $0 \rightarrow 1 \rightarrow 2$ (cost 2); $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ (cost 4).

Source Node 1

Step	N'	$D(0)$	$D(1)$	$D(2)$	$D(3)$
Init	{1}	(1,0)	0	(1,2)	∞
1	{1,0}	(1,0)	0	(1,2)	(8,0)
2	{1,0,2}	(1,0)	0	(1,2)	(3,2)
3	{1,0,2,3}	(1,0)	0	(1,2)	(3,2)

Paths: $1 \rightarrow 0$ (cost 1); $1 \rightarrow 2$ (cost 1); $1 \rightarrow 2 \rightarrow 3$ (cost 3).

Source Node 2

Step	N'	$D(0)$	$D(1)$	$D(2)$	$D(3)$
Init	{2}	(3,0)	(1,1)	0	(2,3)
1	{2,1}	(2,1)	(1,1)	0	(2,3)
2	{2,1,0}	(2,1)	(1,1)	0	(2,3)
3	{2,1,0,3}	(2,1)	(1,1)	0	(2,3)

Paths: $2 \rightarrow 1 \rightarrow 0$ (cost 2); $2 \rightarrow 1$ (cost 1); $2 \rightarrow 3$ (cost 2).

Source Node 3

Step	N'	$D(0)$	$D(1)$	$D(2)$	$D(3)$
Init	{3}	(7,0)	∞	(2,2)	0
1	{3,2}	(5,2)	(3,2)	(2,2)	0
2	{3,2,1}	(4,2)	(3,2)	(2,2)	0
3	{3,2,1,0}	(4,2)	(3,2)	(2,2)	0

Paths: $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ (cost 4); $3 \rightarrow 2 \rightarrow 1$ (cost 3); $3 \rightarrow 2$ (cost 2).

2.1.4 Final Routing Tables - Link-State

	Destination	Next Hop	Cost
Node 0:	1	1	1
	2	1	2
	3	1	4

	Destination	Next Hop	Cost
Node 1:	0	0	1
	2	2	1
	3	2	3

	Destination	Next Hop	Cost
Node 2:	0	1	2
	1	1	1
	3	3	2

	Destination	Next Hop	Cost
Node 3:	0	2	4
	1	2	3
	2	2	2

Q2.2. Part 2 (Optional): Distance-Vector Routing (Bellman-Ford)

2.2.1 Algorithm Description

Distance-Vector routing is **distributed**: each node starts knowing only its own direct link costs, then repeatedly exchanges distance vectors with neighbours until the estimates converge. The update rule at node x for destination y is:

$$D_x(y) = \min_{v \in \text{neighbours}(x)} \{c(x, v) + D_v(y)\}$$

2.2.2 Direct Link Costs Matrix

$$\text{DIRECT} = \begin{pmatrix} 0 & 1 & 3 & 7 \\ 1 & 0 & 1 & \infty \\ 3 & 1 & 0 & 2 \\ 7 & \infty & 2 & 0 \end{pmatrix}$$

2.2.3 Python Implementation

Listing 2: Distance-Vector Routing (Bellman-Ford)

```

1 INF = float('inf')
2
3 DIRECT = [
4     [0, 1, 3, 7],
5     [1, 0, 1, INF],
6     [3, 1, 0, 2],
7     [7, INF, 2, 0],
8 ]
9
10 dt = [[[INF]*4 for _ in range(4)] for _ in range(4)]
11
12 class RtPkt:
13     def __init__(self, sourceid, destid, mincost):
14         self.sourceid = sourceid
15         self.destid = destid

```

```

16         self.mincost = mincost[:]
17
18     event_queue = []
19
20     def tolayer2(pkt):
21         event_queue.append(pkt)
22
23     def min_cost(node):
24         return [min(dt[node][dest]) for dest in range(4)]
25
26     def rtinit(node):
27         neighbors = [j for j in range(4)
28                     if DIRECT[node][j] != INF and j != node]
29         for dest in range(4):
30             for via in range(4):
31                 if via == dest and DIRECT[node][dest] != INF:
32                     dt[node][dest][via] = DIRECT[node][dest]
33                 else:
34                     dt[node][dest][via] = INF
35         dt[node][node] = [0]*4
36         mc = min_cost(node)
37         for nb in neighbors:
38             tolayer2(RtPkt(node, nb, mc))
39
40     def rtupdate(node, pkt):
41         via = pkt.sourceid
42         neighbors = [j for j in range(4)
43                     if DIRECT[node][j] != INF and j != node]
44         changed = False
45         for dest in range(4):
46             new_cost = DIRECT[node][via] + pkt.mincost[dest]
47             if new_cost < dt[node][dest][via]:
48                 dt[node][dest][via] = new_cost
49                 changed = True
50         if changed:
51             mc = min_cost(node)
52             for nb in neighbors:
53                 if nb != via:
54                     tolayer2(RtPkt(node, nb, mc))

```

2.2.4 Initialization - Distance Tables after rtinit

Node 0

dest/via	Node 0	Node 1	Node 2	Node 3	
Node 0	0	0	0	0	Sends [0, 1, 3, 7] to Nodes 1, 2, 3.
Node 1	∞	1	∞	∞	
Node 2	∞	∞	3	∞	
Node 3	∞	∞	∞	7	

Node 1

dest/via	Node 0	Node 1	Node 2	Node 3	
Node 0	1	∞	∞	∞	Sends [1, 0, 1, ∞] to Nodes 0 and 2.
Node 1	0	0	0	0	
Node 2	∞	∞	1	∞	
Node 3	∞	∞	∞	∞	

Node 2

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	3	∞	∞	∞
Node 1	∞	1	∞	∞
Node 2	0	0	0	0
Node 3	∞	∞	∞	2

Sends $[3, 1, 0, 2]$ to Nodes 0, 1, 3.**Node 3**

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	7	∞	∞	∞
Node 1	∞	∞	∞	∞
Node 2	∞	∞	2	∞
Node 3	0	0	0	0

Sends $[7, \infty, 2, 0]$ to Nodes 0 and 2.**2.2.5 Convergence - Final Distance Tables**

After all packets are processed, the final tables are:

Node 0

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	0	0	0	0
Node 1	∞	1	4	10
Node 2	∞	2	3	9
Node 3	∞	4	5	7

Node 1

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	1	∞	3	∞
Node 1	0	0	0	0
Node 2	3	∞	1	∞
Node 3	5	∞	3	∞

Node 2

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	3	2	∞	7
Node 1	4	1	∞	5
Node 2	0	0	0	0
Node 3	7	4	∞	2

Node 3

dest/via	Node 0	Node 1	Node 2	Node 3
Node 0	7	∞	4	∞
Node 1	8	∞	3	∞
Node 2	9	∞	2	∞
Node 3	0	0	0	0

2.2.6 Final Routing Tables - Distance-Vector

Node 0:	Destination	Next Hop	Min Cost
	1	1	1
	2	1	2
	3	1	4
Node 1:	Destination	Next Hop	Min Cost
	0	0	1
	2	2	1
	3	2	3
Node 2:	Destination	Next Hop	Min Cost
	0	1	2
	1	1	1
	3	3	2
Node 3:	Destination	Next Hop	Min Cost
	0	2	4
	1	2	3
	2	2	2

Q2.3. Comparison: Link-State vs. Distance-Vector

Property	Link-State (Dijkstra)	Distance-Vector (Bellman-Ford)
Knowledge	Full global topology at each node	Only direct neighbour costs initially
Message complexity	$O(nE)$ broadcasts across network	Exchanges only with direct neighbours
Convergence	Fast - computed locally after broadcast	Slower - iterative exchange until stable
Routing loops	None (Dijkstra gives optimal paths)	Count-to-infinity problem possible
Final result	Identical optimal routing tables	Identical optimal routing tables

Both algorithms converge to the same minimum-cost paths. The optimal route from Node 0 to Node 3 is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ with total cost $1 + 1 + 2 = 4$, which is cheaper than the direct link cost of 7.

3. Question 3

Setup. A 2×2 input-queued switch with infinitely backlogged queues. Each HOL packet's destination port is chosen independently and uniformly from $\{1, 2\}$.

- If $d_1(t) \neq d_2(t)$: both packets are sent; both queues advance.
- If $d_1(t) = d_2(t)$: one is chosen at random and sent; only that queue advances.

The state $d(t) = (d_1(t), d_2(t))$ forms a Markov chain over $\mathcal{S} = \{11, 12, 21, 22\}$.

Q3.1. Part 1: Transition Matrix and Stationary Distribution

3.1.1 Deriving the Transition Matrix

In each slot, the HOL destinations of both queues are redrawn uniformly from $\{1, 2\}$.

States 12 and 21 ($d_1 \neq d_2$): both packets are sent and both queues advance; the next state is two independent uniform draws:

$$P(\cdot) = \frac{1}{4} \text{ for each of } \{11, 12, 21, 22\}.$$

State 11 ($d_1 = d_2 = 1$): one queue wins at random, its HOL is replaced by a new uniform draw, the other keeps destination 1:

$$P(11 \rightarrow 11) = \frac{1}{2}, \quad P(11 \rightarrow 12) = \frac{1}{4}, \quad P(11 \rightarrow 21) = \frac{1}{4}, \quad P(11 \rightarrow 22) = 0.$$

State 22 ($d_1 = d_2 = 2$): by symmetry,

$$P(22 \rightarrow 11) = 0, \quad P(22 \rightarrow 12) = \frac{1}{4}, \quad P(22 \rightarrow 21) = \frac{1}{4}, \quad P(22 \rightarrow 22) = \frac{1}{2}.$$

$$P = \begin{pmatrix} \frac{1}{2} & \frac{1}{4} & \frac{1}{4} & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ 0 & \frac{1}{4} & \frac{1}{4} & \frac{1}{2} \end{pmatrix} \quad (\text{rows: from } 11, 12, 21, 22)$$

3.1.2 Computing the Stationary Distribution

By the symmetry of the chain, $\pi(11) = \pi(22) \equiv a$ and $\pi(12) = \pi(21) \equiv b$.

Balance equation for state 11:

$$a = \frac{a}{2} + \frac{b}{4} + \frac{b}{4} = \frac{a}{2} + \frac{b}{2} \quad \Rightarrow \quad a = b.$$

Normalisation: $2a + 2b = 1 \Rightarrow 4a = 1 \Rightarrow a = \frac{1}{4}$.

$$\pi(11) = \pi(12) = \pi(21) = \pi(22) = \frac{1}{4}.$$

The chain has a **uniform stationary distribution**.

Verification: $(\pi P)_{11} = \frac{1}{4} \cdot \frac{1}{2} + \frac{1}{4} \cdot \frac{1}{4} + \frac{1}{4} \cdot \frac{1}{4} + \frac{1}{4} \cdot 0 = \frac{1}{4} \checkmark$. All four entries match by symmetry, and $\sum \pi_i = 1 \checkmark$.

Q3.2. Part 2: Average Throughput

3.2.1 Calculation

State	Condition	Packets sent	$T(s)$	$\pi(s)$
11	Collision	1		$\frac{1}{4}$
12	No collision	2		$\frac{1}{4}$
21	No collision	2		$\frac{1}{4}$
22	Collision	1		$\frac{1}{4}$

$$\Lambda = \frac{1}{4}(1 + 2 + 2 + 1) = \frac{6}{4} = \boxed{1.5 \text{ packets/slot}}$$

3.2.2 HOL Blocking Analysis

Metric	Value
Average throughput Λ	1.5 packets/slot
Maximum possible (no HOL blocking)	2.0 packets/slot
Loss due to HOL blocking	0.5 packets/slot
Efficiency	75%

Collision states (11 and 22) occur with combined probability $\frac{1}{2}$, each sending only 1 packet instead of 2. This causes an average loss of $\frac{1}{2} \times 1 = 0.5$ packets/slot, reducing throughput to **75%** of the theoretical maximum.

Q3.3. Python Code and Output

```
1 import numpy as np
2
3 P = np.array([
4     [1/2, 1/4, 1/4, 0],
5     [1/4, 1/4, 1/4, 1/4],
6     [1/4, 1/4, 1/4, 1/4],
7     [0, 1/4, 1/4, 1/2],
8 ])
9
10 # Solve pi @ P = pi
11 A = (P.T - np.eye(4))
12 A[-1, :] = 1
13 b = np.zeros(4); b[-1] = 1
14 pi = np.linalg.solve(A, b)
15
16 packets_per_state = [1, 2, 2, 1]
17 throughput = sum(pi[i]*packets_per_state[i] for i in range(4))
18 # throughput = 1.5 packets/slot
19
20 print(np.allclose(pi, pi @ P)) # True
```

```
pi(11) = pi(12) = pi(21) = pi(22) = 0.25
Throughput (analytical) : 1.500000 packets/slot
Throughput (simulation) : 1.500727 packets/slot
Max possible           : 2.0 packets/slot
Efficiency              : 75.0%
piP == pi              : True
Sum(pi)                : 1.000000
```